SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	A Mohammed Arif	Register Number	192521238
Department / Section	AI & ML	Academic Year / Semester	2025-2026
Faculty Name	Latha R	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	30/10/2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks

8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required

Failure case	P	HTTP 404 test case documented; actual execution record required
Screenshots	P	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required

Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System (Total Marks – 50)

Duration: 1 week

Scenario – Smart Campus Cloud Migration

A multi-campus university currently runs its Student Information System, Learning Management System, Library System, Exam Portal and Email on ageing physical servers in a single on-premise data centre. Each application has its own dedicated server, which stays idle for most of the year but fails to cope during admissions, result publication and online-exam weeks.

The university's current environment is:

- 12 physical servers, average CPU utilisation of about 15%, with peaks above 95% during admissions and exams
- Approximately 25,000 students and 2,000 staff across 4 campuses
- Applications built as large monolithic programs, with no public interfaces for mobile apps or partner colleges
- High capital expenditure, slow procurement (6–8 weeks per server) and no disaster recovery site

As a cloud consultant, you must study how computing evolved to the cloud model, and then design a cloud-based solution that uses virtualization, web services and an appropriate mix of service models (IaaS, PaaS, SaaS and XaaS) for the university. All recommendations must be justified in terms of cost, scalability, availability and manageability.

Design Challenge:

On Day 4, the university announces that it will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, with a requirement of 99.9% availability and a limited operating budget. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

Design and Propose cloud computing model for the context of the university's problems.

- Definition of cloud computing and the five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service)
- Deployment models (public, private, hybrid, community) and which one suits the university
- Three problems of the current on-premise setup and how each cloud characteristic addresses them
- Benefits and limitations of cloud adoption for the university

Evidence Expected

A clear explanation of cloud characteristics and deployment models, mapped to the university's actual problems.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

Trace how computing infrastructure evolved to make cloud computing possible.

- A timeline from mainframes, minicomputers and PCs, to client–server, clusters, grid computing, utility computing and cloud
- Hardware evolution: multi-core processors, commodity servers, high-capacity storage, high-speed networking and data centres
- How each stage solved a limitation of the previous one
- Which stage the university's current system corresponds to, with reasons

Evidence Expected

A labelled timeline or diagram, with a justified explanation of how hardware advances enabled the cloud.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

Explain how internet software evolved and why it supports cloud services.

- Evolution from static web pages to dynamic web, Web 2.0, SOA, mashups and cloud-hosted applications
- Role of browsers, HTTP, AJAX and APIs in delivering software over the internet
- Comparison between desktop software, client–server software and web-delivered software

- How this evolution affects the design of the university's applications

Evidence Expected

A comparative explanation of software evolution stages, linked to the university's application design.

TASK 4 – Apply Server Virtualization (5 Marks)

Apply server virtualization to consolidate the university's 12 physical servers.

- Explain how a hypervisor works, and distinguish Type-1 and Type-2 hypervisors, with examples
- Compare full virtualization, para-virtualization, OS-level virtualization (containers) and hardware-assisted virtualization
- Propose which applications should run as virtual machines and which as containers, with reasons
- Calculate the consolidation: with an average utilisation of 15% per server and a target host utilisation of 70%, estimate how many physical hosts are needed (show your assumptions and working)
- Discuss live migration, snapshots and resource over-commitment as benefits, and the risks of virtualization (noisy neighbour, hypervisor failure, security isolation)

Show your calculation. A final number without working will not receive full credit.

Evidence Expected

A justified virtualization plan with a worked consolidation calculation and discussion of benefits and risks.

TASK 5 – Apply Web Services (4 Marks)

Analyse web services to expose the university's functions to other systems.

- Overview of web services and their role in cloud and SOA
- Compare SOAP, WSDL, UDDI and RESTful services (HTTP methods, JSON/XML)
- Identify at least three university functions to expose as web services (for example, result lookup, fee payment, library search) and define the endpoint, method and sample request/response for each
- Justify whether SOAP or REST is more suitable for each function

Evidence Expected

A defined set of web services with sample request/response formats and a justified choice of SOAP or REST

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

Analyse which service model fits each university application.

- Define IaaS, PaaS and SaaS, and show who manages which layer (hardware to application) in each model
- Categorise the five university applications (SIS, LMS, Library, Exam Portal, Email) into IaaS, PaaS or SaaS, with reasons
- Compare the models on control, cost, scalability, maintenance effort and vendor lock-in
- Give one real provider example for each model

Present your **categorisation in a table**, but explain the **reasoning behind** each **decision**. A table alone is not sufficient.

Evidence Expected

A reasoned mapping of applications to service models, supported by a layer-wise responsibility comparison.

TASK 7 – Examine XaaS (3 Marks)

Examine the broader XaaS (Anything-as-a-Service) concept for the university.

- Explain XaaS and describe at least four variants (for example DaaS, BaaS/MBaaS, FaaS, DBaaS, Security-as-a-Service, Disaster-Recovery-as-a-Service)
- Select two XaaS offerings that would benefit the university, and justify them
- Discuss the risks of over-reliance on multiple 'as-a-Service' providers

Evidence Expected

An analysis of XaaS variants, with justified selection of two offerings for the university.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

Bloom's Level: Level 6 – Create | *Action verbs: Design, Construct, Propose*

Create a complete cloud architecture for the university by combining virtualization, web services and service models.

- An architecture diagram showing users, load balancer, web/application tier, database tier, storage, API layer and DR region
- The deployment model and the service model chosen for each component
- How elasticity (auto-scaling) will handle admissions and exam peaks
- Basic security and availability measures (access control, encryption, backup, multi-zone deployment)

Evidence Expected

A labelled architecture diagram with written justification for each component and design decision.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

Implement a small proof-of-concept of your design using a free-tier cloud account or a local virtualization tool.

- Create a virtual machine or container (using AWS/Azure/GCP free tier, VirtualBox, or Docker) and host a simple web page or REST API for one university function
- Deploy a SaaS or PaaS component (for example a cloud-hosted database, a serverless function or a static site)
- Test the web service with a client (browser, Postman or curl) and record the output
- Provide screenshots, configuration steps and any issues faced

Evidence Expected

Annotated screenshots and clear, reproducible steps showing that the proof-of-concept works.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

Evaluate your design and recommend the final cloud solution for the university.

- Compare on-premise, IaaS-based and PaaS/SaaS-based options on cost (CapEx vs OpEx), scalability, availability, security and effort
- Estimate a simple 3-year cost comparison using stated assumptions
- Identify the main risks (vendor lock-in, data privacy, downtime, skills gap) and give mitigation steps
- Provide your final recommendation and a short migration roadmap (which application moves first, and why)

Your recommendation must be supported by the analysis in the previous tasks. Choosing a popular cloud provider without justification is not sufficient.

Evidence Expected

A technically justified recommendation, supported by a cost comparison, risk analysis and migration roadmap.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, availability must be 99.9%, and the operating budget is limited.

Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Cloud Computing – Introduction & Characteristics	L2 – Understand	3
Task 2	Evolution & Hardware Evolution	L2 – Understand	3
Task 3	Internet Software Evolution	L2 – Understand	3
Task 4	Server Virtualization	L3 – Apply	5
Task 5	Web Services	L3 – Apply	4
Task 6	Service Models: IaaS, PaaS, SaaS	L4 – Analyze	5
Task 7	XaaS	L4 – Analyze	3
Task 8	Design the Cloud Solution Architecture	L6 – Create	5

Task	Title	Bloom's Level	Marks
Task 9	Hands-on Implementation & Documentation	L3 – Apply	4
Task 10	Evaluate & Recommend the Final Solution	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4 / L5 / L6	8
	TOTAL		50

TASK 1 – INTRODUCE CLOUD COMPUTING AND MAP IT TO THE SCENARIO

Meaning and definition

Cloud computing is a model for delivering computing resources—such as servers, storage, databases, networking, platforms and software—over a network when they are required. Instead of buying and maintaining every server inside the university, the institution can obtain resources from a cloud provider and pay according to usage. For the Smart Campus, cloud services can host the Student Information System (SIS), Learning Management System (LMS), Library System, Exam Portal and Email.

Five essential characteristics

Characteristic	Meaning	Application to the university
On-demand self-service	Users or administrators provision resources automatically without waiting for provider staff.	IT staff can create a test server or increase database capacity when admissions begin.
Broad network access	Services are available over standard networks and devices.	Students and staff in four campuses can access portals through browsers and mobile devices.
Resource pooling	Provider resources are shared securely among multiple customers using a multi-tenant model.	Compute, storage and network capacity are pooled; each university application remains logically isolated.
Rapid elasticity	Resources can scale up or down quickly as demand changes.	Extra application instances can be added during result publication and online

Deployment models

Model	Description	Suitability
Public cloud	Infrastructure owned by a provider and delivered to customers over the internet.	Useful for elastic web tiers, development environments and public-facing APIs.
Private cloud	Cloud environment dedicated to one organisation.	Suitable for highly controlled workloads and sensitive legacy systems, but costs more to operate.
Hybrid cloud	Combination of private/on-premise resources and public cloud with controlled integration.	Best initial fit: sensitive records can be retained in a controlled environment while public cloud provides scalable portals,

		backup and APIs.
Community cloud	Infrastructure shared by organisations with common requirements.	Could support affiliated colleges if governance, identity and data-sharing agreements are standardised.

Deployment models

Model	Description	Suitability
Public cloud	Infrastructure owned by a provider and delivered to customers over the internet.	Useful for elastic web tiers, development environments and public-facing APIs.
Private cloud	Cloud environment dedicated to one organisation.	Suitable for highly controlled workloads and sensitive legacy systems, but costs more to operate.
Hybrid cloud	Combination of private/on-premise resources and public cloud with controlled integration.	Best initial fit: sensitive records can be retained in a controlled environment while public cloud provides scalable portals, backup and APIs.
Community cloud	Infrastructure shared by organisations with common requirements.	Could support affiliated colleges if governance, identity and data-sharing agreements are standardised.

Current problems and cloud response

Current problem	Cloud response	Expected result
12 servers average only 15% CPU utilisation	Virtualise and consolidate workloads; use measured capacity.	Better hardware utilisation and lower idle-server cost.
Peak utilisation exceeds 95% during admissions and exams	Auto-scaling and load balancing add capacity during peaks.	Fewer slowdowns and reduced need to purchase peak-only hardware.
Server procurement takes 6–8 weeks; no DR site	Provision resources on demand and replicate backups to a second region.	Faster deployment and improved recovery capability.

Benefits and limitations

- Benefits: lower upfront capital expenditure, flexible capacity, faster provisioning, centralised monitoring, remote access, managed backups and easier integration through APIs.
- Limitations: dependence on internet connectivity, recurring operational charges, possible vendor lock-in, data residency and privacy obligations, provider outages, and the need for cloud-skilled staff.

- Controls: use encryption, role-based access control, budget alerts, portable containers,

TASK 2 – TRACE THE EVOLUTION AND HARDWARE EVOLUTION

Cloud computing became practical through a sequence of changes in computing architecture, hardware capacity, networking and software. Each stage reduced a limitation of the earlier stage and moved computing toward shared, remotely accessible resources.

Evolution of Computing Infrastructure



Hardware evolution that enabled cloud

- Multi-core processors: several processing cores execute workloads in parallel and improve server throughput.
- Commodity servers: standard, relatively inexpensive servers can be combined into large pools rather than relying only on specialised machines.
- High-capacity storage: storage arrays, distributed file systems and object storage hold large volumes of student records, learning material and backups.
- High-speed networking: faster switching and internet links support communication between virtual machines, storage and geographically separated data centres.
- Modern data centres: redundant power, cooling, physical security and network connectivity allow providers to operate infrastructure continuously at scale.

Which stage describes the current university?

The university mainly operates a traditional client–server model hosted in a central on-premise data centre. The evidence is that each of the 12 physical servers is dedicated to an application, while users access those applications over campus networks. It is not yet a mature cloud environment because resources are not pooled and elastically provisioned, procurement is slow, and there is no disaster-recovery site. The migration should therefore move the institution from dedicated client–server infrastructure toward virtualised hybrid cloud.

TASK 3 – ANALYSE INTERNET SOFTWARE EVOLUTION

Stage	Description	University relevance
Static web pages	Mainly fixed HTML pages delivered by a web server.	Suitable for notices and public information, but not interactive student transactions.
Dynamic web	Server-side programs generate pages using databases and user input.	Supports login, course registration, fee status and result display.
Web 2.0	Interactive, collaborative and responsive web applications.	Supports LMS discussion, online submissions and student dashboards.
Service-Oriented Architecture (SOA)	Application functions are exposed as reusable, loosely coupled services.	SIS, library and fee functions can be reused by portals and mobile apps.
Mashups	Combines information or functions from multiple services.	A campus dashboard may combine timetable, library availability and fee status.
Cloud-hosted applications	Applications run on scalable cloud infrastructure and are accessed through networks.	Provides elastic access across four campuses and affiliated colleges.

Browser, HTTP, AJAX and APIs

- Browser: the user interface used to access the university portal without installing a separate desktop program.
- HTTP/HTTPS: request-response protocols used to transfer pages and API data; HTTPS adds encryption through TLS.
- AJAX: allows a page to request data asynchronously and update part of the screen without reloading the entire page.
- API: a documented interface through which systems exchange data. REST APIs commonly use HTTP methods and JSON.
-

Software delivery comparison

Feature	Desktop software	Client–server software	Web-delivered software
Installation	Installed on each computer.	Client program may need installation.	Usually accessed through a browser or app.
Updates	Update each device.	Client and server updates may both be needed.	Central server-side updates simplify maintenance.
Access	Often tied to a device or local network.	Usually network-dependent.	Accessible across locations using secure internet access.
Scaling	Limited by local device.	Limited by server capacity and architecture.	Can use load balancing and cloud auto-scaling.

University fit	Legacy specialist applications.	Existing SIS and library architecture.	Preferred interface for mobile-friendly, multi-campus services.

Design impact: the university should gradually separate monolithic functions into well-defined services. A common identity service, API gateway and shared data contracts reduce duplication. Legacy modules can remain temporarily behind adapters while newer functions are developed as web services.

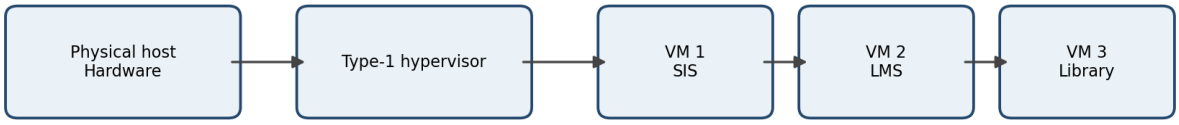
TASK 4 – APPLY SERVER VIRTUALIZATION

How a hypervisor works

A hypervisor, also called a virtual machine monitor, allocates physical CPU, memory, storage and network resources to multiple virtual machines. Each VM receives virtual hardware and can run its own guest operating system. The hypervisor schedules CPU time, maps memory, manages virtual disks and isolates workloads.

Type	Operation	Examples / use
Type-1 (bare-metal)	Runs directly on physical server hardware.	VMware ESXi, Microsoft Hyper-V, KVM- based platforms; preferred for production
		data-centre consolidation.
Type-2 (hosted)	Runs as an application on a host operating system.	Oracle VirtualBox, VMware Workstation; useful for student labs, testing and demonstrations.

Server Virtualization and Consolidation



Virtualization approaches

Approach	How it works	Strength / limitation
Full virtualization	Hypervisor presents complete virtual hardware to an unmodified guest OS.	Good compatibility; may add some overhead.
Para-virtualization	Guest OS is aware of the hypervisor and uses hypercalls for selected operations.	Can improve efficiency; guest support or modification may be required.
OS-level virtualization (containers)	Multiple isolated user spaces share the host OS kernel.	Lightweight and fast; containers do not provide a separate kernel for each application.
Hardware-assisted virtualization	CPU features such as Intel VT-x or AMD-V assist the hypervisor.	Improves execution of virtualised workloads; still requires correct configuration.

Consolidation calculation

Given: 12 physical servers; average CPU utilisation = 15% per server; target host utilisation = 70%. Assume each original server has equal CPU capacity, and the workload can be consolidated without a major performance penalty.

Total workload expressed in server-capacity units = 12 × 15% = 1.8 full-server equivalents.

Required host capacity at the target utilisation = 1.8 ÷ 0.70 = 2.571 hosts. Since a fraction of a host cannot be deployed, round up: 3 physical hosts are the theoretical minimum.

Validation and safety: three hosts provide only 2.1 server-capacity units at 70% utilisation. This is a planning estimate based on average CPU only. Memory, disk IOPS, network throughput, licensing, high availability and peak CPU above 95% must also be measured. For production resilience, a practical starting design may use 4 hosts: three can carry normal consolidated load while the fourth provides headroom/maintenance capacity. A proper sizing exercise must use peak and percentile utilisation, not averages alone.

Benefits and risks

- Live migration moves a running VM between compatible hosts with minimal interruption, supporting maintenance and balancing.
- Snapshots capture a VM state for short-term rollback before changes; they are not a substitute for independent backups.
- Resource over-commitment allows assigned virtual CPU or memory to exceed physical capacity, improving average utilisation but risking contention.
- Noisy-neighbour risk occurs when one workload consumes disproportionate shared CPU, memory, disk or network resources. Apply quotas, reservations and monitoring.
- Hypervisor failure can affect many VMs. Use redundant hosts, clustering, patching and tested recovery.
- Security isolation can fail if hypervisor vulnerabilities or misconfigured virtual networks expose workloads. Apply least privilege, segmentation and timely updates.

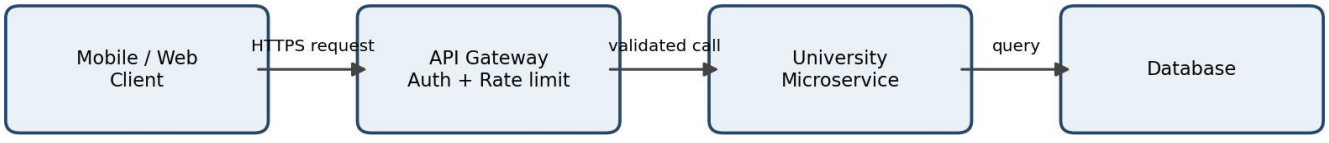
TASK 5 – APPLY WEB SERVICES

A web service exposes a software function through a standard network interface so another application can request it. In a Service-Oriented Architecture, services are loosely coupled and reusable. For the university, web services allow the mobile app, student portal and affiliated-college systems to access approved functions without directly connecting to internal databases.

Technology	Purpose	Key points
SOAP	XML-based messaging protocol.	Formal message envelope, standards and enterprise extensions; can support strict contracts and transaction/security requirements.
WSDL	Web Services Description Language.	XML document describing SOAP service operations, messages and endpoints.
UDDI	Service registry concept.	Directory for publishing and discovering services; less common in many modern REST deployments.
REST	Architectural style using resources and standard HTTP.	Commonly uses GET, POST, PUT/PATCH and DELETE; JSON is widely used, though XML is also possible.

SOAP may be appropriate where an existing partner system requires a strict WSDL contract, formal XML schema validation or WS-Security. For the proposed mobile and college integrations, REST is generally simpler and lighter. Payment processing must never expose card details or trust client-supplied payment status; the server should verify the payment provider .

Example API Request and Response Flow



Common API controls

- Use HTTPS/TLS for all requests and encrypt sensitive data at rest.
- Authenticate users and partner colleges using OAuth 2.0 / OpenID Connect or another approved identity mechanism.
- Apply role-based authorisation, scopes and tenant-specific permissions.
- Validate input, use parameterised database queries and return safe error messages.
- Use rate limits, request-size limits, audit logs, API versioning and monitoring.

TASK 6 – ANALYSE IaaS, PaaS AND SaaS

IaaS provides virtualised compute, storage and networking. PaaS provides a managed application runtime and development platform. SaaS provides a complete application that users access as a service.

Layer	IaaS – customer manages	PaaS – customer manages	SaaS – customer manages
Physical data centre / hardware	Provider	Provider	Provider
Virtualisation / OS	Customer manages guest OS and patches; provider manages physical virtualisation	Provider manages platform OS and runtime	Provider manages
Middleware / runtime	Customer	Provider	Provider
Application code / configuration	Customer	Customer application code and configuration	Mostly tenant configuration and use
Data and user access	Customer	Customer	Customer remains responsible for data governance, access settings and lawful use

Mapping the five university applications

Application	Proposed model	Reason
SIS	IaaS initially, then selected modules on PaaS	Legacy monolith may need OS-level control and compatibility; modern modules can move to managed runtimes.
LMS	PaaS or SaaS	A managed learning platform reduces infrastructure work; PaaS is useful if extensive customisation is required.
Library System	IaaS initially / PaaS after modernisation	Preserves legacy integrations first; later can move search and catalogue services to managed application services.

Exam Portal	PaaS	Managed runtime and auto-scaling support short periods of high traffic; exam integrity controls remain university responsibility.
Email	SaaS	Email, spam filtering, availability and updates are efficiently managed by a specialist provider.

Model comparison

Factor	IaaS	PaaS	SaaS
Control	Highest control over OS and network configuration.	Control over application code, less control over platform.	Lowest technical control; configuration and data governance remain.
Cost	Compute/storage charges plus IT administration.	Managed platform charges; less operations effort.	Subscription per user or usage.
Scalability	Flexible but must configure scaling.	Often includes managed scaling features.	Provider scales the service; customer has limited control.
Maintenance	High: OS, runtime, middleware and app.	Medium: app and data; provider manages platform.	Low infrastructure maintenance.
Vendor lock-in	Can be reduced with standard VM images.	Runtime and managed services may create lock-in.	Data formats, workflows and integrations may create lock-in.

Provider examples: IaaS – Amazon EC2, Microsoft Azure Virtual Machines or Google Compute Engine; PaaS – Azure App Service, Google App Engine or AWS Elastic Beanstalk; SaaS – Microsoft 365, Google Workspace or a hosted LMS. These are examples, not a final procurement selection.

TASK 7 – EXAMINE XaaS

XaaS means “Anything-as-a-Service”. It describes technology capabilities delivered as subscription or usage-based services instead of being installed and maintained entirely by the customer.

Variant	Meaning	University use
DaaS – Desktop as a Service	Cloud-hosted virtual desktops.	Provide controlled desktops for temporary staff, labs or remote administration.
BaaS / MBaaS – Backend / Mobile Backend as a Service	Managed backend services for mobile applications.	Authentication, push notifications and application data services for the student mobile app.
FaaS – Function as a Service	Run event-triggered functions without managing servers.	Send result notifications or process approved file-upload events.
DBaaS – Database as a Service	Managed database engine, backups and patching.	Host student and library databases with encryption, replication and backup.
Security-as-a-Service	Managed security capabilities such as WAF, threat detection and identity.	Protect public portals and monitor suspicious access.
DRaaS – Disaster Recovery as a Service	Provider-supported replication and recovery environment.	Recover critical services if the primary region becomes unavailable.

Two recommended offerings

1. DBaaS: a managed relational database can reduce routine database patching and backup work. It should be configured with private networking, encryption, high availability, automated backups, point-in-time recovery and strict access policies. The university must test restore procedures and retain ownership of data classification and access decisions.

2. DRaaS: the scenario explicitly states that there is no disaster-recovery site. A managed recovery environment can provide replicated backups and documented recovery procedures without requiring the university to build a second full data centre. Define recovery time objective (RTO) and recovery point objective (RPO), then test failover and restore

Risks of depending on multiple providers

- Different identity, logging and billing systems increase complexity.
- Service outages or provider changes can affect multiple dependent applications.
- Data transfer and egress charges may increase total cost.
- Different contracts and data locations complicate compliance.

- Mitigation: standard APIs, central identity, central monitoring, data export tests, service-level agreements, exit clauses, cost alerts and documented fallback procedures.

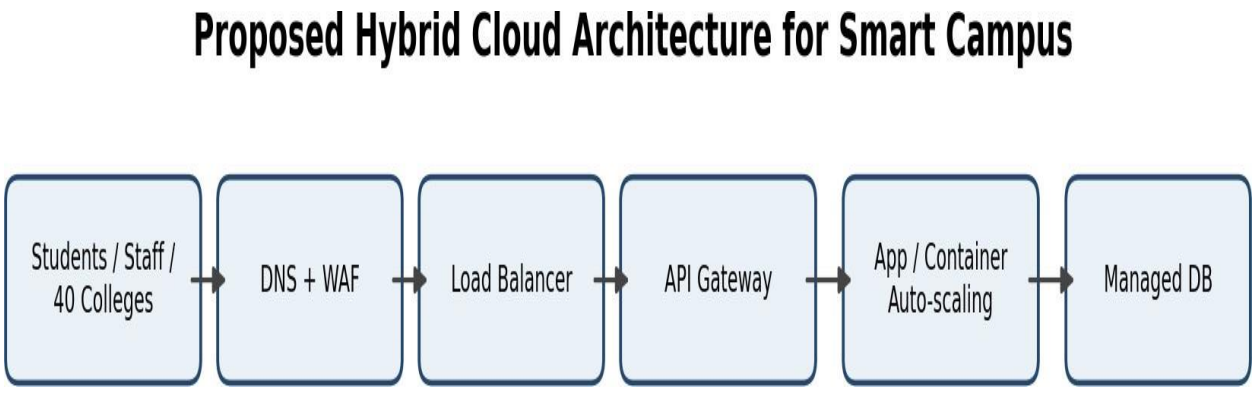
Risks of depending on multiple providers

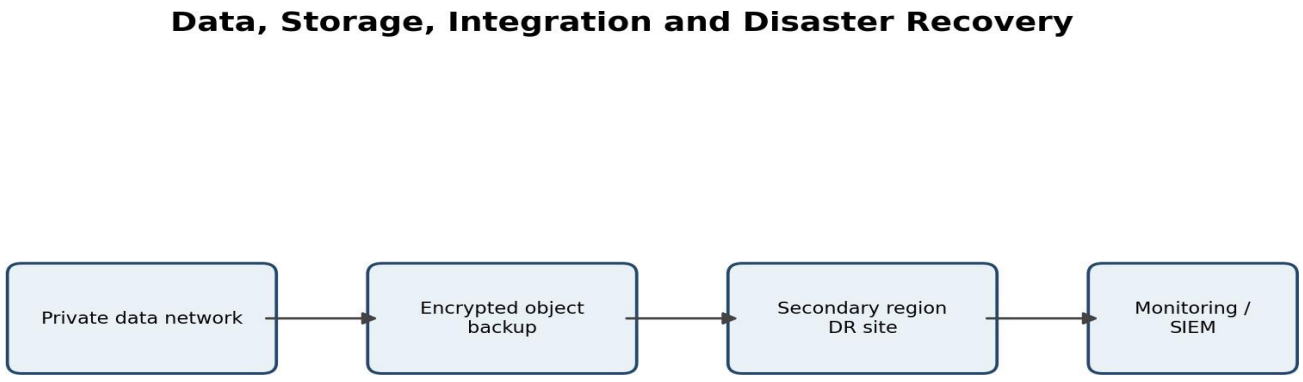
- Different identity, logging and billing systems increase complexity.
- Service outages or provider changes can affect multiple dependent applications.
- Data transfer and egress charges may increase total cost.
- Different contracts and data locations complicate compliance.
- Mitigation: standard APIs, central identity, central monitoring, data export tests, service-level agreements, exit clauses, cost alerts and documented fallback procedures.

TASK 8 – DESIGN THE CLOUD SOLUTION ARCHITECTURE

Set minimum application instances for normal demand and configure automatic scale-out based on CPU, request rate, queue depth and response time

- Use a load balancer and health checks so failed instances stop receiving traffic.
- Cache non-sensitive catalogue and static content close to users; do not cache private student records without strict controls.





Component explanation and service model

Component	Model / deployment	Purpose and justification
Users and campus networks	Client devices; internet/VPN	Students, staff and authorised partner-college users access approved services.
DNS, WAF and DDoS protection	Security-as-a-Service / public cloud	Routes traffic and filters common web attacks before application entry.
Load balancer	Managed cloud service	Distributes requests across healthy instances and avoids a single application server bottleneck.
API gateway	PaaS / managed API service	Central authentication, authorisation, rate limiting, versioning and request routing.
Web/application tier	PaaS or container platform	Runs SIS adapters, LMS integration, library services and exam portal; scales horizontally.
Database tier	DBaaS in private network	Stores transactional records with encryption, backups and high availability.
Object storage	IaaS storage service	Stores documents, learning materials, exports and backup copies.
DR region	DRaaS / secondary region	Maintains recoverable copies and a standby environment for major outages.

Scaling strategy

- Set minimum application instances for normal demand and configure automatic scale-out based on CPU, request rate, queue depth and response time.
- Use a load balancer and health checks so failed instances stop receiving traffic.
- Cache non-sensitive catalogue and static content close to users; do not cache private student records without strict controls.
- Use database connection pooling, read replicas for read-heavy queries and queue-based processing for non-urgent notifications.
- Load-test admission and exam scenarios before production and set maximum scaling limits to protect the budget.

Security, availability and disaster recovery

- Identity: single sign-on where possible, multi-factor authentication for staff and privileged accounts, and least-privilege roles.
- Network: private subnets for databases, firewall rules, network segmentation and restricted administrative access.
- Data: TLS in transit, encryption at rest, managed keys, data classification and audit trails.

- Availability: deploy critical components across at least two availability zones, monitor health and use tested failover.
- Backup: automated database backups, immutable or protected copies, separate credentials and regular restore tests.
- Disaster recovery: replicate critical data to a secondary region; document RTO/RPO and conduct scheduled recovery exercises.

Deployment recommendation: hybrid cloud. Public cloud provides scalable front-end services and managed platform components. Sensitive databases should be isolated in private network segments with access limited to approved services. A private/on-premise component may be retained temporarily where legacy dependencies or policy require it.

TASK 9 – HANDS-ON IMPLEMENTATION AND DOCUMENTATION

This section provides a reproducible proof-of-concept based on the assignment scenario. The source assignment marks execution evidence as partly present; therefore the commands and outputs below are an implementation guide and expected results, not a claim that the application has already been run. Insert your own genuine terminal/browser screenshots and actual output before submission.

9.1 Proof-of-concept: Student Result REST API using Flask and Docker

The API demonstrates one university function: result lookup by register number. It runs in a Docker container, representing OS-level virtualisation. The same API can later be deployed to a managed PaaS runtime.

Step 1 – Create project files

Create a folder named university-result-api. Inside it, create app.py, requirements.txt and Dockerfile.

app.py

```
from flask import

Flask, jsonify app =

Flask(_name_)

RESULTS = {
   "1925260102": {
      "student": "Sample
      Student",
      "semester": 2,
      "status":
      "published",
      "subjects": [
         {"name": "Cloud Computing", "marks": 86},
         {"name": "Python Programming", "marks": 90}
      ]
   }
}

@app.get("
/") def
home():
```

```
   return jsonify({"message": "University Result API is running"})

@app.get("/api/v1/results
/<reg_no>") def
get_result(reg_no):
   result =
   RESULTS.get(reg_no)
   if result is None:
      return jsonify({"error": "Student result
   not found"}), 404 return jsonify({"regNo":
   reg_no, **result}), 200

if __name__== "__main__":
   app.run(host="0.0.0.0"
   , port=5000)
```

requirements.txt

```
Flask==3.0.3
```

Dockerfile

```
FROM python:3.12-
slim WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r
requirements.txt COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 2 – Build and run the container

```
docker build -t university-result-api .

docker run --name result-api -p 5000:5000 university-result-api

docker ps
```

The first command builds an image. The second starts the container and maps host port 5000 to container port 5000. The third lists running containers. If Docker is not installed, install Docker Desktop or use a local Linux environment with Docker Engine.

Step 3 – Test using a browser or curl

```
curl http://localhost:5000/

curl

http://localhost:5000/api/v1/results/1925260102

curl -i

http://localhost:5000/api/v1/results/UNKNOWN
```

Test	Expected result	What it verifies
GET /	HTTP 200; JSON message that API is running.	Container and Flask web server are reachable.
GET /api/v1/results/1925260102	HTTP 200; JSON student result.	Successful result lookup.
GET /api/v1/results/UNKNOWN	HTTP 404; JSON error message.	Correct handling of missing student record.

Step 4 – PaaS deployment option

For a PaaS demonstration, create a web application in a provider service such as Azure App Service, Google App Engine or AWS Elastic Beanstalk. Connect the source repository or upload the application, configure the Python runtime and start command, set environment variables, deploy, and test the public HTTPS endpoint. Provider screens and exact menus may change; follow the current provider console instructions.

Screenshot evidence to add

- Screenshot 1: project folder showing app.py, requirements.txt and Dockerfile.
- Screenshot 2: successful Docker image build in terminal.
- Screenshot 3: docker ps showing the running container.
- Screenshot 4: browser or Postman showing HTTP 200 and JSON result.
- Screenshot 5: unknown register number showing HTTP 404.
- Screenshot 6: PaaS deployment page and public HTTPS endpoint, if deployed.

Evidence note: use actual screenshots from your own system. Do not paste illustrative or fabricated screenshots as proof. Record the operating system, Docker version, date, commands, endpoint and any errors faced.

TASK 10 – EVALUATE AND RECOMMEND THE FINAL SOLUTION

Three-way comparison

Factor	Traditional on-premise	IaaS-based cloud	PaaS/SaaS-based cloud
Cost model	High CapEx for servers, facilities and refresh; staff costs continue.	Mainly OpEx usage charges, plus OS and operations effort.	Usage/subscription OpEx; lower infrastructure administration.
Scalability	Procurement lead time of 6–8 weeks; fixed capacity.	Can provision VMs quickly; scaling must be configured.	Managed scaling features can reduce operational delay.

Availability , security and Effort

Availability	Depends on local data centre and its redundancy; no DR site currently.	Can deploy across zones, but university configures much of it.	Provider manages platform availability subject to service terms.
Security	Direct physical control but university manages all layers.	More control; shared responsibility remains.	Less infrastructure control; careful configuration and data governance required.
Effort	High hardware, patching, backup and facility effort.	Medium to high: OS and application operations remain.	Lower infrastructure effort; integration and vendor management remain.

Illustrative three-year cost comparison

The assignment does not provide actual provider quotations, server purchase prices, staff salaries or usage rates. The following is an illustrative planning model in Indian rupees, not a vendor quote. Replace the assumptions with procurement quotations before making a financial commitment.

Cost category	On-premise (₹ lakh / 3 years)	IaaS (₹ lakh / 3 years)	PaaS/SaaS (₹ lakh / 3 years)
Hardware / refresh / setup	30	3	1
Compute, storage and network	6	30	24
Software / licences / subscriptions	9	6	15
Operations and administration	18	12	6
Backup, DR and security	9	6	6
Illustrative total	72	57	52

Calculation: total three-year cost is the sum of the five categories. On-premise = 30 + 6 + 9 + 18 + 9 = ₹72 lakh. IaaS = 3 + 30 + 6 + 12 + 6 = ₹57 lakh. PaaS/SaaS = 1 + 24 + 15 + 6 + 6 = ₹52 lakh. Under these assumptions, PaaS/SaaS has the lowest estimated three-year total, while IaaS offers more control. Actual totals may change with student count, storage growth, data

egress, licence terms, discounts and peak traffic.

Main risks and mitigation

Risk	Impact	Mitigation
Vendor lock-in	Migration to another provider may be difficult.	Use open APIs, containers, exportable data formats, infrastructure-as-code and exit clauses.
Data privacy	Unauthorised access or improper data handling.	Data classification, encryption, least privilege, audit logs and contractual controls.
Downtime	Students may lose access during exams or admissions.	Multi-zone deployment, monitoring, failover, backups and tested DR procedures.
Skills gap	Cloud services may be misconfigured or poorly managed.	Staff training, standard operating procedures, code review and managed services where justified.
Unexpected bills	Traffic spikes or unbounded resources increase cost.	Budgets, alerts, quotas, autoscaling limits, tagging and monthly cost reviews.

Phased migration roadmap

Phase	Activities	Reason / exit condition
1. Discover and prepare	Inventory applications, dependencies, data classification, peak metrics, RTO/RPO and network requirements.	Baseline is documented; owners approve target architecture.
2. Foundation	Create cloud landing zone, identity federation, network segmentation, logging, keys, budgets and backup policy.	Security and governance controls are tested.
3. Low-risk pilot	Move static website or a non-critical library search/API component.	Validate connectivity, monitoring, rollback and cost.
4. Modernise and migrate	Containerise selected LMS and Exam	Performance and functional tests pass.

	Portal components; migrate databases with validation.	
5. SIS migration	Move SIS in controlled waves with data reconciliation, parallel run and rollback plan.	Business owners sign off on accuracy and availability.
6. Optimise and retire	Tune scaling, remove unused servers, test DR and decommission legacy hardware.	Costs, capacity and recovery meet agreed targets.

Final recommendation: choose a hybrid architecture with PaaS for new web services and the exam-facing application tier, SaaS for email, managed database services where suitable, and IaaS for legacy workloads that cannot yet be refactored. This balances control, scalability, migration risk and operating effort.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT

Impact analysis

The new requirement changes the system from a university-only service into a partner ecosystem. A mobile application is being introduced, student-record and fee-payment services will be exposed to 40 affiliated colleges, expected load becomes ten times the current peak, availability must be 99.9%, and the operating budget remains limited.

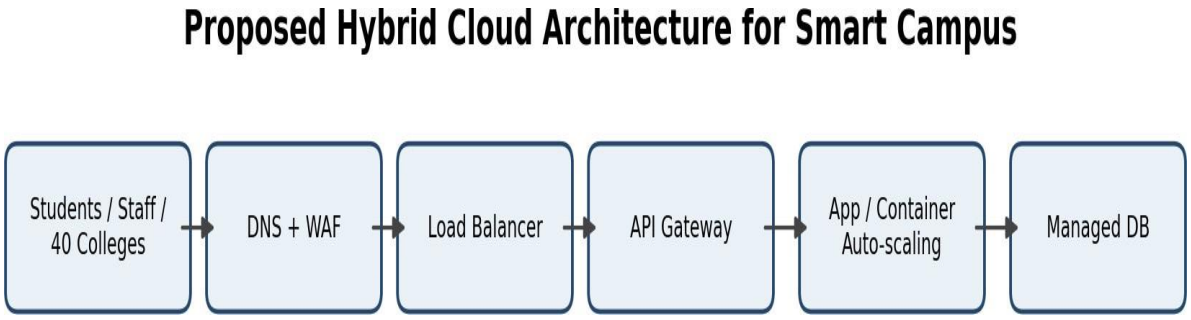
- Load assumption changes from current peak to 10× current peak. Average CPU utilisation of 15% is no longer a sufficient sizing basis.
- The number and type of users increase: external college users need controlled access to only their permitted students and operations.
- APIs become critical because the mobile app and college systems must communicate through stable, versioned and secure interfaces.
- Multi-tenant access creates a risk of one college viewing another college's student records. Every request must be scoped to an authenticated college tenant.
- 99.9% availability allows approximately 0.1% annual downtime: 365 × 24 × 0.001 = 0.876 hours, or about 52.6 minutes per year. This equals 8.76 hours (about 8 hours and 46 minutes) of permitted downtime per year.

Evaluate the original design

The original hybrid design is a sound starting point but should not be retained unchanged. Three physical hosts may satisfy the average CPU consolidation calculation, but it does not prove capacity for a tenfold peak or provide sufficient fault tolerance. A single-region design is not adequate for a strong recovery target because a regional outage could affect all services. Auto-scaling helps the application tier, but the database, API gateway, identity provider and payment integration can become bottlenecks.

The revised plan should retain the hybrid-cloud foundation but modify the application and integration tiers. Use API gateway protection, stateless containerised services, queue-based asynchronous processing, multi-zone deployment and a secondary recovery region. Keep the database tier private and scale it using tested database sizing, connection pooling and read replicas where appropriate.

Revised architecture



- API gateway: authenticate each college and mobile user, validate scopes, apply quotas and rate limits, and record audit events.
- Tenant isolation: issue a unique tenant identifier for each affiliated college; derive tenant identity from trusted authentication claims, never from an unverified client field. Enforce tenant filters in service and database layers.
- Container orchestration: deploy stateless services across multiple availability zones and scale replicas using CPU, request rate and queue depth.
- Payment processing: use a payment provider's hosted/tokenised flow, signed callbacks, idempotency keys and reconciliation jobs. Do not store raw card data.
- Database: use private networking, encryption, high availability, point-in-time recovery and tenant-aware access controls.
- Multi-region recovery: replicate backups and critical data to a secondary region; define which services are warm standby and which can be restored on demand.
-

Capacity and availability defence

Capacity method: measure the current peak request rate (P) and resource consumption under a representative load test. The new design target is at least 10P requests per second, plus agreed headroom. If one tested application instance safely serves R requests per second at the target latency, the theoretical instance count is N = ceiling(10P ÷ R). Add failure headroom so that the remaining instances can still serve demand after one instance or zone is lost. Do not size from the 15% historical average alone.

Availability: deploy across at least two availability zones and remove single points of failure in the load balancer, application tier and database. A 99.9% annual service objective allows about

52.6 minutes of downtime per year. Multi-zone deployment reduces zone-level risk, while a secondary region and tested recovery procedures address regional incidents. The exact achieved availability depends on the end-to-end service and provider SLAs; individual component SLAs should not simply be added.

Cost comparison and trade-offs

The revised design will cost more than the original low-load design because it requires additional application replicas, API management, security monitoring, database capacity, cross-zone traffic and recovery copies. To protect the limited budget, scale out only when metrics justify it, use serverless functions for bursty even. processing, cache safe read-only content, set per-tenant quotas, schedule non-production resources to stop, and review storage retention. Compare the incremental monthly cost of the 10× peak design against the cost of lost service during admissions and exams.

Decision area	Original design	Day-4 revised design
Application hosting	VMs and some containers	Stateless containers with orchestration and autoscaling; serverless for selected event tasks
API exposure	Basic REST APIs	Managed API gateway, versioning, authentication, quotas and tenant policies
Availability	Single-region / basic backup	Multi-zone production plus secondary-region recovery
Security	User access control and encryption	Per-college tenant isolation, stronger API controls, audit and anomaly monitoring
Cost control	Usage monitoring	Autoscaling bounds, per-tenant quotas, budgets, caching and scheduled shutdowns

Final decision

Decision: MODIFY the original design rather than retain it unchanged or rebuild everything. The hybrid foundation, use of managed services and phased migration remain appropriate. The mobile and affiliated-college requirements require a stronger API layer, tenant isolation, container orchestration, multi-zone availability and a secondary-region recovery plan. This approach limits migration disruption while directly addressing the new requirements.

CONCLUSION

The university's dedicated on-premise servers are underused on average but overloaded during admissions and examinations. Cloud computing, virtualisation, web services and suitable service models provide a path to better utilisation, faster provisioning and improved access. The proposed hybrid-cloud solution combines IaaS for legacy compatibility, PaaS for modern application services, SaaS for email and XaaS offerings such as DBaaS and DRaaS. The Day-4 challenge shows that the architecture must evolve to support tenfold peak demand, secure access by 40 colleges and a 99.9% availability objective. A phased migration, genuine implementation evidence, measured capacity tests and regular cost/security reviews are essential.

REFERENCES

- Assignment 1 question paper: CSA15 – Cloud Computing and Big Data Analytics, "Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System" (13-page source document supplied with this task).
- NIST Special Publication 800-145, The NIST Definition of Cloud Computing.
- Docker Documentation – Get started / Dockerfile reference (for container build and run concepts).
- Flask Documentation – Quickstart and API reference (for the sample REST API).
- Cloud provider documentation for the selected IaaS, PaaS, SaaS, database, API gateway and disaster-recovery services. Consult the current provider documentation before deployment.